API Security with Tyk: Session Objects

Learn how Tyk handles identity, access, and lifecycle through session objects.

Tyk

What is a Session Object?

- In Tyk, every identity maps to a Session Object.
- Supported identity types:
  - Bearer Tokens
  - HMAC Keys
  - JSON Web Tokens (JWT)
  - OpenID Connect (OIDC) identities
  - Basic Auth users
- Think of it as the user’s metadata — how Tyk knows who is accessing what and how.

What Does a Session Object Contain?

A session object defines:

- Rate limit rules
- Quota limits
- Access Control List (ACL)
- Policy ID (if used)
- Expiry time of the access
- Optional metadata and alias for identification

Where are Session Objects Stored?

- Stored in Redis (not MongoDB or Gateway memory).
- Stored as:

```
<hashed_token_string> : <session_object_JSON>
```

- Tokens are hashed for security.
- Use Alias to identify sessions in logs and analytics.

API Keys

- Underlying data structure for keys stored in Tyk (in Redis)
- Every auth type in the gateway will result in a structure like this existing under the hood.
- Can be hashed with a number of algorithms i.e. murmur 64/128, sha1/256 or not at all.


```
{
  "last_check": 0,
  "allowance": 1000,
  "rate": 1000,
  "per": 1,
  "expires": 1458669677,
  "quota_max": 1000,
  "quota_renews": 1458667309,
  "quota_remaining": 1000,
  "quota_renewal_rate": 3600,
  "access_rights": {
    "e1d21f942ec746ed416ab97fe1bf07e8": {
      "api_name": "Closed",
      "api_id": "e1d21f942ec746ed416ab97fe1bf07e8",
      "versions": ["Default"],
      "allowed_urls": null
    }
  },
  "org_id": "53ac07777cbb8c2d53000002",
  "oauth_client_id": "",
  "basic_auth_data": {
    "password": "",
    "hash_type": ""
  },
  "jwt_data": {
    "secret": ""
  },
  "hmac_enabled": false,
  "hmac_string": "",
  "is_inactive": false,
  "apply_policy_id": "",
  "apply_policies": [
    "59672779fa4387000129507d",
    "53222349fa4387004324324e",
    "543534s9fa4387004324324d"
    ],
  "data_expires": 0,
  "monitor": {
    "trigger_limits": null
  },
  "meta_data": {
    "test": "test-data"
  },
  "tags": ["tag1", "tag2"],
  "alias": "john@smith.com"
}
```

Key Fields Explained (1/2)

Field	Description
`rate` / `per`	Defines rate limiting window (e.g., 1000 req/sec)
`quota_max`	Max requests in a quota period
`quota_remaining`	Remaining quota
`quota_renewal_rate`	How long the quota lasts (e.g., 3600 = 1h)
`access_rights`	Defines which APIs/versions are accessible
`org_id`	Organization that owns the token

Key Fields Explained (2/2)

Field	Description
`apply_policies`	List of attached policy IDs
`meta_data`	Custom info for transforms/middleware
`tags`	Tags added to analytics
`alias`	Human-readable identifier
`is_inactive`	If true → denies access
`expires`	UNIX timestamp for key expiry

Tyk

Metadata & Alias

Metadata

- Key/value pairs for user context (e.g., user_id, email, plan_type)
- Can be injected into:
  - Headers
  - URL rewrites
  - Request/Response transforms
  - Virtual Endpoints
- Used for personalization, validation, or upstream enrichment.

Alias

- Human-readable identifier for a token
- Appears in logs and analytics (important because tokens are hashed in Redis)

Session Metadata in Action

Metadata can be used by:

- URL Rewrite
- Header Transformations
- Body Transformations
- Virtual Endpoints
- Custom Plugins

Example use case:

Inject `meta_data.user_email` into a header for upstream logging.

Expiry vs Invalidation

Term	Meaning
Expiry	Token remains in Redis but is no longer valid — returns `401 Key has expired`.
Invalidation / Deletion	Token is deleted from Redis — returns `400 Access disallowed`.

Important:

Developers who hardcode tokens may need expiry to notify renewal instead of full invalidation.

Key Lifetime and Expiry

Tyk distinguishes between:

- Expiry: token can't be used (but still in Redis)
- Lifetime: how long token stays in Redis before deletion

Controlling Key Lifetime
You can control key deletion at:

1. API level → `session_lifetime` in API definition
2. Gateway level → `global_session_lifetime` in `tyk.conf`

Note: a `0` value means infinite lifetime (never deleted).

Lifetime Example

Example:

```
"session_lifetime": 86400
```

→ Token deleted from Redis after 24 hours of creation.

⚠️ Use `session_lifetime_respects_key_expiration: true` to prevent early deletion.

Key Lifetime Precedence Summary

force_global_session_lifetime	session_lifetime_respects_key_expiration	Effective Lifetime
true	true	global_session_lifetime
true	false	global_session_lifetime
false	true	larger of session_lifetime or expires
false	false	session_lifetime

Recap

- Session Object = Identity + Access Metadata
- Stored in Redis (hashed key → JSON object)
- Controls rate limit, quota, ACL, expiry, and policy
- Metadata enhances flexibility and customization
- Separate expiry and deletion behavior
- Key lifetime configurable at API or Gateway level

Tyk